ShopFlask

eCommerce Project Documentation

Built with Flask · SQLite · Jinja2 · Chart.js

Flask	SQLite	Jinja2	Chart.js	Werkzeug

PBL College Project · 2025

1. Project Overview

ShopFlask is a full-stack eCommerce web application built as a Project-Based Learning (PBL) exercise. It demonstrates the complete flow of a real-world shopping platform — from user registration and product browsing to cart management, checkout, and order tracking — all implemented with beginner-friendly Python and Flask.

1.1 Objectives

- Build a working eCommerce application from scratch using Flask
- Implement secure user authentication with hashed passwords
- Design a relational SQLite database with foreign key relationships
- Create a session-based cart system without a dedicated cart table
- Build an admin panel with real-time sales analytics using Chart.js
- Deploy the project live on Render (free tier)

1.2 Tech Stack

Technology	Purpose
Python 3	Core programming language
Flask 3.0	Web framework — routing, sessions, templating
SQLite	Lightweight relational database (built into Python)
Jinja2	HTML templating engine (built into Flask)
Werkzeug	Password hashing (generate_password_hash / check_password_hash)
Chart.js	JavaScript charting library for admin dashboard graphs
Gunicorn	Production WSGI server for Render deployment
CSS / JS	Plain vanilla CSS and JavaScript — no heavy frameworks

2. Folder Structure

The project follows a modular Blueprint-based structure. Each feature has its own route file, keeping the codebase organized and easy to navigate.

```
ecommerce/
  app.py              ← Application factory + home route
  database.py         ← DB connection helpers (get_db, init_db)
  schema.sql          ← SQL CREATE TABLE statements
  seed.py             ← Insert sample products (run once locally)
  render_setup.py     ← Auto DB init for Render deployment
```

```
decorators.py       ← @login_required, @admin_required
Procfile            ← Gunicorn start command
requirements.txt    ← Python dependencies
routes/
  auth.py           ← /register, /login, /logout
  products.py       ← /products, /product/<id>
  cart.py           ← /cart, add, update, remove
  orders.py         ← /checkout, /orders, tracking, recommendations
  admin.py          ← /admin/* dashboard, orders, products
templates/
  base.html         ← Shared layout (navbar, flash, footer)
  home.html, auth/, products/, cart/, orders/, admin/
static/
  css/style.css     ← All custom styles
  js/main.js        ← Flash message auto-dismiss
instance/
  ecommerce.db      ← SQLite database file (auto-created)
```

3. Database Design

The database has four tables linked by foreign keys. SQLite is used because it requires no installation or configuration — the database is a single file stored in the instance/ folder.

3.1 Tables

Table	Columns	Purpose
users	id, username, email, password_hash, role, created_at	Stores customer and admin accounts
products	id, name, description, price, stock, category, image_url	All products in the store
orders	id, user_id (FK), total_price, status, created_at	Each purchase placed by a user
order_items	id, order_id (FK), product_id (FK), quantity, price	Line items inside each order

3.2 Relationships

- One user can place many orders (users → orders, 1-to-many)
- One order contains many order items (orders → order_items, 1-to-many)
- One product can appear in many order items (products → order_items, 1-to-many)
- order_items is the junction table connecting orders and products

> Why store price in order_items?
> Product prices can change over time. By saving the price at the moment

> of purchase inside order_items, past order history always stays accurate.

3.3 Order Status Flow

pending → **processing** → **shipped** → **delivered** / **cancelled**

---

4. app.py — Application Factory

The Application Factory pattern is used — instead of creating the Flask app at the module level, it is created inside a create_app() function. This is the industry-standard approach because it makes the app easier to test and configure.

4.1 What create_app() does

1. Creates the Flask app instance
2. Sets SECRET_KEY and DATABASE path from environment variables
3. Creates the instance/ folder if it does not exist
4. Registers the database teardown (close_db)
5. Imports and registers all five Blueprints (auth, products, cart, orders, admin)
6. Defines the home route (/) which queries and renders the first 8 products
7. Registers the flask init-db CLI command

4.2 Environment Variables

Variable	Where used
SECRET_KEY	Flask session encryption — must be a strong random string in production
DATABASE	Path to ecommerce.db inside instance/ folder
RENDER	Set to "true" on Render to trigger render_setup.py auto-init

---

5. database.py — DB Connection

database.py provides three helper functions used throughout the project. Flask's g object (request context global) is used to store a single database connection per request.

Function	What it does	When called
get_db()	Returns the current DB connection, creating one if needed	Inside every route that queries the DB

close_db()	Closes the DB connection at the end of each request	Automatically by Flask teardown
init_db()	Reads schema.sql and creates all tables	Via flask init-db CLI command
init_app()	Registers close_db with the Flask app	Once inside create_app()

> sqlite3.Row is set as row_factory so that query results behave like dictionaries. Instead of row[0], you can write row["username"] which makes template code and Python code much more readable.

6. Routes — Blueprint Files

All routes are split into Blueprint files inside the routes/ folder. Each Blueprint handles one feature area. This keeps app.py clean and makes each feature easy to find and edit independently.

6.1 routes/auth.py — Authentication

Route	Method
/register	GET/POST
/login	GET/POST
/logout	GET

- register: Validates form input, checks for duplicate email, hashes the password using generate_password_hash(), inserts the new user, and redirects to login.
- login: Fetches the user by email, verifies the password with check_password_hash(), then stores user_id, username, and user_role in the Flask session.
- logout: Calls session.clear() to wipe all session data, then redirects to login.

> Passwords are NEVER stored as plain text.
> Werkzeug's generate_password_hash() applies bcrypt-style hashing.
> check_password_hash() compares the hash safely without reversing it.

6.2 routes/products.py — Product Pages

Route	What it shows
/ and /products	All products with optional ?category= filter. Fetches distinct categories for the filter bar.

/product/<id>	Single product detail. Also fetches 4 related products from the same category.

6.3 routes/cart.py — Shopping Cart

The cart is stored entirely in Flask's session — no database table is needed until checkout. The cart structure is a dictionary of product_id → {name, price, quantity, image_url}.

Route	Action
/cart	Display all cart items and the running total
/cart/add/<id>	Add product to cart, or increase quantity if already present. Caps at stock limit.
/cart/update/<pid>	Update quantity. Setting quantity to 0 removes the item.
/cart/remove/<pid>	Remove a single item from the cart.
/cart/clear	Wipe the entire cart from the session.

> session.modified = True must be called after changing a mutable object (like a dict) inside the session. Without it, Flask may not save the change because it only detects reassignment, not in-place mutation.

6.4 routes/orders.py — Orders & Checkout

Route	Action
/checkout	GET shows order summary. POST validates stock, creates order row, inserts order_items, reduces stock, clears cart.
/orders	Order history for the logged-in user, sorted newest first.
/orders/<id>	Detail view of one order with all items (JOIN with products).
/orders/<id>/track	Visual progress bar showing current status step.
/recommendations	Products in categories the user has ordered, excluding already-ordered products. Falls back to popular products.

6.5 routes/admin.py — Admin Panel

All admin routes require the @admin_required decorator. The user's role must be "admin" in the database. The admin blueprint uses url_prefix="/admin" so all routes are under /admin/*.

Route	Action
/admin/	Dashboard: summary cards, Chart.js line + doughnut charts, top products, recent orders.
/admin/orders	Full order list with status filter. Inline status update form.
/admin/orders/<id>/update	POST — updates order status in DB.
/admin/products	Product list with stock warning (red if stock < 5).
/admin/products/add	Form to add a new product with all fields.
/admin/products/<id>/delete	POST — deletes a product.

7. decorators.py — Route Protection

Two decorator functions protect routes from unauthorized access. They are applied with @ syntax above any route function, just like @app.route.

Decorator	What it checks	On failure
@login_required	Is user_id present in session?	Redirect to /login with flash warning
@admin_required	Is user logged in AND role == admin?	Redirect to /products with flash error

> Decorators use functools.wraps(f) to preserve the original function name.
> Without @wraps, Flask would see multiple routes all named "decorated_function"
> and raise an AssertionError about duplicate endpoint names.

8. Templates — Jinja2 Frontend

All HTML files use Jinja2 templating. Every page extends base.html which provides the navbar, flash messages, and footer. Child templates only define their unique content inside {% block content %}.

8.1 base.html

- Renders the sticky navbar with dynamic links based on session state
- Shows the logged-in username, cart item count, and admin link (if admin)
- Renders flash messages with color-coded alert classes (success, danger, info, warning)
- Loads style.css and main.js via url_for("static", ...)
- Contains {% block content %} where child pages inject their HTML

8.2 Key Jinja2 Features Used

Syntax	What it does
{{ variable }}	Output a Python variable into HTML
{% if session.user_id %}	Conditional — show/hide navbar links based on login status
{% for product in products %}	Loop over a list of database rows
{% extends "base.html" %}	Inherit the base layout
{% block content %}	Define the section that child templates fill in
"%.2f"|format(price)	Format a float to 2 decimal places (e.g. 1499.00)
url_for("auth.login")	Generate URLs by blueprint name + function name

9. Static Files

9.1 static/css/style.css

A single CSS file styles the entire application. Key sections include:

- .navbar — sticky top nav with dark background and red accent logo
- .product-grid — CSS Grid with auto-fill columns that wraps responsively
- .product-card — white card with hover lift effect (transform + box-shadow)
- .auth-card — centered login/register form with shadow
- .product-detail — 2-column grid (image left, info right), collapses on mobile
- .btn, .btn-primary, .btn-dark, .btn-outline — reusable button styles
- .alert-* — color-coded flash message boxes
- @media (max-width: 768px) — responsive breakpoint for mobile

9.2 static/js/main.js

A small script that auto-dismisses flash messages after 4 seconds using a CSS opacity transition followed by DOM removal. This prevents the page from feeling cluttered after actions like "Added to cart".

10. Cart Flow — Step by Step

This is the core business logic of the application. Understanding this flow is important for the viva.

8. User opens a product detail page and clicks "Add to Cart"

9. A POST request goes to /cart/add/<product_id>
10. Flask fetches the product from the database to get name, price, stock
11. The cart is read from session["cart"] (a dict). If missing, an empty dict is created
12. If the product is already in the cart, quantity is increased (capped at stock)
13. If new, a new entry is added: { name, price, quantity, image_url }
14. session.modified = True tells Flask to save the updated session
15. User goes to /cart — the cart dict is read and rendered, total is calculated in Python
16. User goes to /checkout — stock is re-validated for each cart item before saving
17. A new row is inserted into orders, then one row per item into order_items
18. Each product's stock column is reduced by the purchased quantity
19. session.pop("cart") clears the cart, user is redirected to their new order

11. Admin Dashboard — Charts & Analytics

The admin dashboard at /admin fetches aggregated data using SQL GROUP BY queries and passes it to the template as Python lists. Chart.js on the frontend converts these lists into interactive charts.

Chart	Type	Data source
Sales - Last 7 Days	Line chart	orders grouped by DATE(created_at), last 7 days
Revenue by Category	Doughnut	order_items JOINed with products, grouped by category
Top Selling Products	Table	SUM(quantity) from order_items, top 5 by units sold
Recent Orders	Table	Last 10 orders with customer username JOIN

The Jinja2 template passes Python lists to Chart.js using the tojson filter, which safely converts Python data to valid JavaScript arrays:

```
const salesLabels = {{ sales_data | map(attribute="day") | list | tojson }};
const revenue     = {{ sales_data | map(attribute="revenue") | list | tojson }};
```

12. Recommendation System

The recommendation system uses a simple content-based filtering approach using only SQL — no machine learning is required.

Strategy	Logic	Used when

1. History	Find categories the user has ordered before	User has at least one past order
2. Filter	Show products from those categories NOT yet ordered by the user	Combined with Strategy 1
3. Popular	Show products ordered most across all users (ORDER BY SUM qty)	User has no order history

13. Deployment on Render

13.1 Key Files

File	Purpose
requirements.txt	Lists flask, werkzeug, gunicorn so Render can pip install them
Procfile	Tells Render to start the app with: gunicorn app:app
render_setup.py	Auto-creates and seeds the DB on first deploy (triggered by RENDER env var)
.gitignore	Excludes instance/, venv/, __pycache__/ from being pushed to GitHub

13.2 Render Settings

Setting	Value
Runtime	Python 3
Build Command	pip install -r requirements.txt
Start Command	gunicorn app:app
SECRET_KEY	Any strong random string (set as env variable)
RENDER	true (triggers render_setup.py)

Important: Render's free tier spins down after 15 minutes of inactivity.
The first request after a sleep period may take 30-60 seconds to respond.
The SQLite database resets on each new deploy. For persistent data,
use Render's paid persistent disk or migrate to PostgreSQL.

14. Complete URL Reference

URL	Access	Description
/	Public	Home page with featured products
/products	Public	All products with category filter
/product/<id>	Public	Product detail + add to cart form
/register	Public	Create a new account
/login	Public	Login with email and password
/logout	Logged in	Clear session and redirect to login
/cart	Logged in	View and manage cart
/checkout	Logged in	Confirm and place order
/orders	Logged in	Order history list
/orders/<id>	Logged in	Single order detail with items
/orders/<id>/track	Logged in	Visual order tracking progress bar
/recommendations	Logged in	Personalised product suggestions
/admin	Admin only	Dashboard with charts
/admin/orders	Admin only	All orders + status update
/admin/products	Admin only	Product list + delete
/admin/products/add	Admin only	Add new product form

15. Quick Start (Local)

20. git clone <your-repo-url> then cd flask-ecommerce
21. python -m venv venv then activate it
22. pip install -r requirements.txt
23. flask init-db
24. python seed.py
25. python app.py
26. Open http://127.0.0.1:5000 in your browser

> Always run flask init-db before python seed.py.
> init-db creates the tables. seed.py inserts the products.
> If you run init-db again, it wipes the database — re-run seed.py after.

16. Common Issues & Fixes

Problem	Cause	Fix
No products on home page	seed.py not run after init-db	Run flask init-db then python seed.py
ModuleNotFoundError	Virtual env not activated	Run venv\Scripts\activate (Windows) or source venv/bin/activate (Mac)
flask: command not found	Flask not installed	pip install flask inside the active venv
Admin page says "access denied"	User role is not "admin"	Run the admin upgrade Python one-liner with your email
TemplateNotFound error	Template file missing or misspelled	Check the templates/ folder and filename
session data lost on restart	SECRET_KEY changed between runs	Use a fixed SECRET_KEY in development

ShopFlask — PBL Project Documentation

Built with Flask · SQLite · Jinja2 · Chart.js · Deployed on Render

2025 · PBL College Project